

Angular pour développeur .NET FullStack

Guide complet • Parties 1 à 12

Angular pour développeur .NET FullStack - Guide complet

Angular pour développeur .NET - Partie 1

PARTIE 1 — COMPRENDRE ANGULAR AVANT DE CODER

1. Pourquoi Angular existe ?

Angular est un framework frontend permettant de créer des applications web complexes sans manipuler le DOM manuellement.

Il organise le code en composants réutilisables et automatise les mises à jour de l'interface.

2. Framework vs Bibliothèque

Une bibliothèque est un outil que tu utilises quand tu le souhaites.

Un framework impose une structure et des règles à suivre.

Angular est un framework.

3. Pourquoi les entreprises aiment Angular ?

- Structure standardisée
- Maintenance facilitée
- Travail en équipe simplifié
- Projets plus cohérents

4. Angular et C#

Les développeurs .NET retrouvent beaucoup de concepts familiers :

- Classes
- Services
- Injection de dépendances
- HttpClient
- Typage fort

5. Comment Angular démarre ?

Cycle :

localhost:4200

→ index.html

→ main.ts

→ bootstrapApplication()

→ AppComponent

→ affichage final

6. Le rôle de main.ts

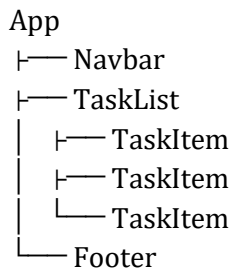
main.ts est le point d'entrée de l'application.

Exemple :

```
bootstrapApplication(App)
```

Cette instruction indique à Angular quel composant utiliser comme racine.

7. L'arbre de composants



Chaque composant possède sa logique, son affichage et son style.

8. CSR, SSR et SSG

CSR :

Le navigateur construit la page.

SSR :

Le serveur construit la page avant de l'envoyer.

SSG :

Les pages sont générées à l'avance.

Résumé :

Angular est un framework frontend structuré basé sur des composants.

Il permet de créer des SPA modernes, maintenables et performantes.

Angular pour développeur .NET - Partie 2

TypeScript pour développeurs C#

Pourquoi Angular utilise TypeScript ?

TypeScript est une surcouche de JavaScript.

Il ajoute des types afin de détecter les erreurs avant l'exécution.

Pour un développeur C#, TypeScript est beaucoup plus naturel que JavaScript pur car il apporte un typage similaire.

Les types primitifs

string : texte

number : nombre

boolean : vrai ou faux

Exemple :

```
let name: string = "Liskara";
```

```
let age: number = 34;
```

```
let isStreamer: boolean = true;
```

Les tableaux

```
let names: string[] = ["Liskara", "Nyx"];
```

```
let scores: number[] = [10,20,30];
```

Le type entre [] indique le type contenu dans le tableau.

Les objets avec type

```
type User = {  
  id:number;  
  name:string;  
};
```

```
const user: User = {  
  id:1,  
  name:"Liskara"  
};
```

Type vs Interface

Les deux servent à décrire la structure d'un objet.

type :

```
type User = { id:number; }
```

```
interface :  
interface User { id:number; }
```

Dans Angular moderne, les deux sont utilisés.

Fonctions typées

```
function add(a:number,b:number):number{  
  return a+b;  
}
```

:number après les parenthèses indique le type retourné.

Classes TypeScript

```
class User {  
  name:string = "";  
}
```

Très proche du C#.

public private readonly

public : accessible partout.

private : accessible uniquement dans la classe.

readonly : valeur non réassignable.

Null et Undefined

undefined = aucune valeur définie.

null = valeur vide volontaire.

Toujours vérifier les valeurs potentiellement nulles.

TypeScript et Angular

Angular utilise TypeScript dans :

- les composants
- les services
- les modèles
- les routes

Comprendre TypeScript facilite énormément l'apprentissage d'Angular.

Comparaison C# / TypeScript

C#

```
string name;
```

```
TypeScript  
let name:string;
```

```
C#  
List<string>
```

```
TypeScript  
string[]
```

Les concepts sont très proches.

Exercice 1

Créer un type Task avec :

- id:number
- title:string
- done:boolean

Créer ensuite une variable respectant ce type.

Correction

```
type Task = {  
  id:number;  
  title:string;  
  done:boolean;  
}
```

```
const task: Task = {  
  id:1,  
  title:"Apprendre Angular",  
  done:false  
};
```

Pièges fréquents

- Oublier les types.
- Mélanger string et number.
- Utiliser any partout.
- Ne pas vérifier les valeurs nulles.

Résumé

TypeScript est l'une des plus grandes forces d'Angular.

Pour un développeur .NET, c'est souvent la partie la plus facile à assimiler.

Angular pour développeur .NET - Partie 3

Les Composants Angular

Qu'est-ce qu'un composant ?

Un composant est une partie réutilisable de l'interface.

Exemples :

- Navbar
- Footer
- Carte produit
- Liste des tâches
- Profil utilisateur

Angular construit une application en assemblant plusieurs composants comme des LEGO.

L'analogie LEGO

Imagine un château LEGO.

Tu ne construis pas tout d'un seul bloc.

Tu assembles plusieurs pièces.

Angular fonctionne exactement de la même manière.

App

- └─ Navbar
- └─ Sidebar
- └─ Content
- └─ Footer

Créer un composant

Commande Angular :

`ng g c task-item`

ou

`ng generate component task-item`

Angular crée automatiquement :

- task-item.ts

- task-item.html
- task-item.css

Les 3 fichiers d'un composant

task-item.ts

→ logique

task-item.html

→ affichage

task-item.css

→ style

Chaque fichier possède une responsabilité précise.

Le décorateur @Component

Exemple :

```
@Component({  
  selector: 'app-task-item',  
  standalone: true,  
  templateUrl: './task-item.html',  
  styleUrls: ['./task-item.css']  
})
```

Ce décorateur indique à Angular comment utiliser le composant.

Le selector

Le selector définit le nom de la balise HTML.

selector : 'app-task-item'

Utilisation :

```
<app-task-item></app-task-item>
```

C'est comparable à une balise HTML personnalisée.

Le template

Le template correspond à l'affichage.

task-item.html

```
<p>{{ title }}</p>
```


Tout ce qui est visible à l'écran se trouve dans le template.

Le style

Le fichier CSS contrôle l'apparence.

```
.task {  
  border: 1px solid black;  
}
```

La logique ne doit jamais être placée dans le CSS.

Standalone Components

Avant Angular 15, il fallait utiliser des NgModules.

Aujourd'hui :

standalone:true

Le composant est autonome.

C'est la méthode moderne.

Les imports

Pour utiliser un composant dans un autre :

```
imports:[TaskItem]
```

Sans cela Angular ne reconnaît pas la balise.

Structure d'une application

App

```
├─ Navbar  
├─ TaskList  
│   ├─ TaskItem  
│   ├─ TaskItem  
│   └─ TaskItem  
└─ Footer
```

Chaque composant possède sa responsabilité.

Comparaison avec C#

Une classe C# représente un objet.

Un composant Angular représente une partie d'interface.

C# :

User

Angular :

TaskItemComponent

Le principe de séparation des responsabilités reste identique.

Bonnes pratiques

Un composant doit :

- être petit
- être réutilisable
- avoir une responsabilité unique

Éviter les composants de plusieurs milliers de lignes.

Exercice

Créer un composant UserCard.

Afficher :

- Nom
- Email
- Téléphone

Puis l'utiliser dans App.

Correction

user-card.ts

```
export class UserCard {  
  name = 'Liskara';  
  email = 'test@test.com';  
}
```

user-card.html

```
<p>{{ name }}</p>  
<p>{{ email }}</p>
```

app.html

```
<app-user-card></app-user-card>
```

Erreurs fréquentes

- Oublier standalone:true
- Oublier imports
- Utiliser un mauvais selector
- Mélanger logique et présentation
- Créer des composants trop gros

Résumé

Un composant est la brique fondamentale d'Angular.

Chaque composant possède :

- logique
- affichage
- style

Une application Angular est un arbre de composants reliés entre eux.

Angular pour développeur .NET - Partie 4

Le Binding : le lien entre TypeScript et HTML

Introduction au Binding

Le Binding est l'un des concepts les plus importants d'Angular.

Il permet de relier les données TypeScript à l'interface HTML.

Sans Binding :

Tu modifies une variable mais l'écran ne change pas automatiquement.

Avec Angular :

La vue se met à jour automatiquement.

Pourquoi le Binding existe ?

En JavaScript classique, il faut souvent utiliser :

```
document.getElementById()  
querySelector()  
innerHTML
```

Angular évite ces manipulations manuelles.

Tu décris simplement ce que tu veux afficher.

Les 3 types de Binding

Angular utilise principalement :

1. Interpolation
2. Property Binding
3. Event Binding

Il faut absolument connaître les trois.

Interpolation

Syntaxe :

```
{{ variable }}
```

Exemple TypeScript :

```
title = 'Ma première tâche';
```

Exemple HTML :

```
<p>{{ title }}</p>
```

Résultat :

Ma première tâche

Comment fonctionne l'interpolation ?

Angular lit la variable TypeScript.

Puis il remplace automatiquement les accolades par la valeur.

C'est la manière la plus simple d'afficher une donnée.

Property Binding

Syntaxe :

```
[attribut]='valeur'
```

Exemple :

```
imageUrl = 'avatar.png';
```

```
<img [src]='imageUrl'>
```

Angular injecte automatiquement la valeur dans l'attribut HTML.

Exemple avec disabled

TypeScript :

```
isDone = true;
```

HTML :

```
<button [disabled]='isDone'>
```

Valider

```
</button>
```

Si isDone vaut true :

Le bouton est désactivé.

Pourquoi utiliser [] ?

Sans crochets :

```
src='imageUrl'
```

Angular considère cela comme du texte.

Avec crochets :

```
[src]='imageUrl'
```

Angular lit la variable.

Event Binding

Syntaxe :

```
(click)='maFonction()'
```

Angular écoute un événement utilisateur.

Exemple complet

TypeScript :

```
onDelete(){  
  console.log('Suppression');  
}
```

HTML :

```
<button (click)='onDelete()'>  
  Supprimer  
</button>
```

Les événements courants

click

change

input

keyup

keydown

submit

Tous peuvent être écoutés avec () .

Récapitulatif visuel

`{{ variable }}`

Afficher une valeur

`[property]`

Envoyer une valeur vers le HTML

`(event)`

Réagir à une action utilisateur

Comparaison avec C#

En WinForms ou WPF on manipule souvent directement les contrôles.

Angular préfère la liaison de données.

Le principe se rapproche davantage du Data Binding de WPF.

Exercice 1

Créer une variable title.

Afficher cette variable avec interpolation.

Exercice 2

Créer une variable isDisabled.

Utiliser Property Binding pour désactiver un bouton.

Exercice 3

Créer une méthode onClick().

Déclencher cette méthode avec Event Binding.

Correction

```
title = 'Angular';
```

```
<p>{{ title }}</p>
```

```
isDisabled = true;
```

```
<button [disabled]='isDisabled'>
```

```
onClick(){  
  console.log('Bonjour');
```

}

<button (click)='onClick()'

Erreurs fréquentes

Oublier les accolades.

Utiliser [] alors qu'on veut afficher du texte.

Utiliser () alors qu'on veut envoyer une valeur.

Confondre les trois syntaxes.

Astuce de mémorisation

{ } = afficher

[] = envoyer

() = écouter

Cette règle permet de retrouver rapidement la bonne syntaxe.

Résumé

Le Binding relie TypeScript et HTML.

Interpolation :

{{ valeur }}

Property Binding :

[property]

Event Binding :

(event)

Ces trois mécanismes sont utilisés dans pratiquement toutes les applications Angular.

Angular pour développeur .NET - Partie 5

Les Directives Modernes : @if, @for et @switch

Pourquoi les directives existent ?

Dans une application réelle, on ne veut pas toujours afficher les mêmes éléments.

Parfois il faut :

- afficher un message uniquement sous certaines conditions ;
- afficher une liste d'objets ;
- afficher un contenu différent selon une valeur.

Les directives Angular servent à gérer ces situations directement dans le HTML.

Qu'est-ce qu'une directive ?

Une directive est une instruction donnée à Angular.

Elle indique comment le HTML doit être affiché.

Les directives modernes sont :

- @if
- @for
- @switch

Elles remplacent progressivement les anciennes syntaxes *ngIf et *ngFor.

La directive @if

@if permet d'afficher un bloc uniquement si une condition est vraie.

Exemple :

```
@if(isAdmin){  
  <p>Bienvenue administrateur</p>  
}
```

Si isAdmin vaut false, le texte n'est pas affiché.

Le @else

Comme en C#, @if peut être accompagné d'un @else.

```
@if(isConnected){  
  <p>Connecté</p>  
}
```

```
@else{  
  <p>Non connecté</p>  
}
```

Un seul bloc sera affiché.

Analogie avec C#

C# :

```
if(isAdmin)  
{  
  Console.WriteLine("Admin");  
}
```

Angular :

```
@if(isAdmin){  
  <p>Admin</p>  
}
```

Le principe est exactement le même.

La directive @for

@for permet d'afficher une collection.

Exemple TypeScript :

```
users = [  
  {id:1,name:'Liskara'},  
  {id:2,name:'Nyx'}  
];
```

Exemple HTML :

```
@for(user of users; track user.id){  
  <p>{{ user.name }}</p>  
}
```

Pourquoi utiliser @for ?

Sans @for il faudrait écrire chaque élément à la main.

Avec @for Angular parcourt automatiquement le tableau.

Le mot-clé track

Le track est extrêmement important.

Exemple :

```
@for(user of users; track user.id)
```

Angular utilise cette valeur pour savoir quel élément a changé.

Toujours utiliser une clé unique.

Le bloc @empty

@empty permet d'afficher un contenu si la liste est vide.

```
@for(user of users; track user.id){  
  <p>{{ user.name }}</p>  
}  
@empty{  
  <p>Aucun utilisateur</p>  
}
```

La directive @switch

@switch permet de gérer plusieurs cas.

Exemple :

```
@switch(priority){  
  
  @case('low'){  
    <p>Faible</p>  
  }  
  
  @case('medium'){  
    <p>Moyenne</p>  
  }  
  
  @case('high'){  
    <p>Haute</p>  
  }  
  
  @default{  
    <p>Inconnue</p>  
  }  
}
```

```
}  
}
```

Comparaison avec C#

C# :

```
switch(priority)  
{  
  case "high":  
    break;  
}
```

Angular :

```
@switch(priority){  
  @case("high"){  
  }  
}
```

Quand utiliser @if ?

Pour afficher ou masquer un élément.

Exemples :

- utilisateur connecté
- rôle administrateur
- chargement terminé

Quand utiliser @for ?

Pour afficher :

- une liste de tâches
- une liste d'utilisateurs
- une liste de produits
- une liste de commentaires

Quand utiliser @switch ?

Lorsque plusieurs états sont possibles.

Exemples :

- priorité
- statut
- rôle utilisateur

Exercice 1

Créer une variable isPremium.

Afficher :

'Compte Premium'

uniquement si la variable vaut true.

Exercice 2

Créer un tableau de jeux vidéo.

Afficher tous les jeux avec @for.

Exercice 3

Créer une variable status.

Afficher :

- Actif
- Inactif
- Suspendu

avec @switch.

Correction

```
@if(isPremium){  
  <p>Compte Premium</p>  
}
```

```
@for(game of games; track game.id){  
  <p>{{ game.name }}</p>  
}
```

```
@switch(status){  
  @case('active'){  
    <p>Actif</p>  
  }  
}
```

Erreurs fréquentes

Oublier track.

Confondre @if et @for.

Utiliser @switch pour seulement deux cas.

Ne pas prévoir le cas @empty.

Lien avec DevTask

Dans le projet DevTask :

@if

→ afficher tâche terminée ou non

@for

→ afficher la liste des tâches

@switch

→ afficher le niveau de priorité

Résumé

@if :

Afficher sous condition

@for :

Afficher une collection

@switch :

Afficher plusieurs cas possibles

Ces trois directives sont utilisées quotidiennement dans Angular moderne.

Angular pour développeur .NET - Partie 6

Les Signals : le cœur d'Angular moderne

Pourquoi Angular a créé les Signals ?

Avant les Signals, Angular devait vérifier régulièrement si quelque chose avait changé dans l'application.

Plus une application grossissait, plus cette vérification devenait coûteuse.

Les Signals ont été créés pour rendre les mises à jour plus simples, plus rapides et plus prévisibles.

Le problème des variables classiques

Exemple :

```
count = 0;
```

```
count = count + 1;
```

La variable change, mais elle ne sait pas qu'elle a changé.

Angular doit donc vérifier toute l'application pour trouver ce qui a été modifié.

L'idée derrière un Signal

Un Signal est une valeur qui sait lorsqu'elle change.

Elle peut prévenir automatiquement Angular.

Ainsi Angular met à jour uniquement les parties concernées de l'écran.

Analogie : la sonnette

Variable classique :

Tu dois vérifier toutes les portes de la maison pour savoir si quelqu'un est arrivé.

Signal :

La sonnette sonne directement lorsqu'une personne arrive.

Angular sait immédiatement où agir.

Créer un Signal

Import :

```
import { signal } from '@angular/core';
```

Création :

```
count = signal(0);
```

Ici le Signal contient la valeur 0.

Lire un Signal

C'est l'erreur la plus fréquente.

Une variable classique se lit ainsi :

```
count
```

Un Signal se lit ainsi :

```
count()
```

Les parenthèses sont obligatoires.

Pourquoi count() ?

Un Signal est un objet spécial.

count représente le Signal.

count() représente la valeur contenue dans le Signal.

Afficher un Signal dans le HTML

TypeScript :

```
count = signal(0);
```

HTML :

```
<p>{{ count() }}</p>
```

Angular affichera la valeur actuelle du Signal.

Modifier un Signal avec set()

set() remplace complètement la valeur.

```
count.set(10);
```

Le Signal contient maintenant 10.

Modifier un Signal avec update()

update() est utilisé lorsque la nouvelle valeur dépend de l'ancienne.

```
count.update(value => value + 1);
```

Très pratique pour les compteurs.

set() ou update() ?

Utiliser set() lorsque tu connais directement la nouvelle valeur.

Utiliser update() lorsque la nouvelle valeur dépend de l'ancienne.

Exemple complet de compteur

```
count = signal(0);
```

```
increment(){  
  count.update(v => v + 1);  
}
```

```
reset(){  
  count.set(0);  
}
```

Les Signals avec des tableaux

Un Signal peut contenir un tableau.

```
tasks = signal([  
  { id:1, title:'Angular' }  
]);
```

Le tableau devient lui aussi réactif.

Ajouter un élément

```
tasks.update(current => [  
  ...current,  
  { id:2, title:'Signals' }  
]);
```

```
]);
```

On crée un nouveau tableau.

Erreur fréquente avec push()

À éviter :

```
tasks().push(newTask);
```

Angular risque de ne pas détecter correctement la modification.

Toujours utiliser set() ou update().

Computed

Computed permet de créer une valeur calculée.

```
remainingCount = computed(() =>  
  tasks().length  
);
```

La valeur est recalculée automatiquement.

Analogie du Computed

Imagine une calculatrice.

Tu modifies les nombres.

Le résultat se met à jour automatiquement.

Tu ne modifies jamais directement le résultat.

Signal vs Computed

Signal :

Valeur modifiable.

Computed :

Valeur calculée automatiquement.

Un Computed ne possède pas de set() ni de update().

Comparaison avec C#

Signal ressemble à une propriété observable.

Computed ressemble à une propriété calculée.

Les concepts sont proches de certains mécanismes MVVM utilisés en WPF.

Exercice 1

Créer un Signal count initialisé à 0.

Afficher sa valeur.

Exercice 2

Créer un bouton qui incrémente count avec update().

Exercice 3

Créer un Computed qui affiche le double du compteur.

Correction

```
count = signal(0);
```

```
doubleCount = computed(() =>  
    count() * 2  
);
```

```
increment(){  
    count.update(v => v + 1);  
}
```

Lien avec DevTask

Dans DevTask :

```
tasks = signal([...])
```

```
remainingCount = computed(...)
```

Quand une tâche est ajoutée :

- tasks change
- remainingCount se recalcule
- l'écran se met à jour automatiquement

Schéma complet

Utilisateur clique



Signal modifié



Angular est prévenu



Computed recalculé



HTML mis à jour

Erreurs fréquentes

Oublier les parenthèses `count()`.

Utiliser `push()` directement.

Modifier un `Computed`.

Confondre `Signal` et variable classique.

Résumé

`signal(valeur)`

Créer un `Signal`

`signal()`

Lire un `Signal`

`signal.set(valeur)`

Remplacer la valeur

`signal.update(...)`

Modifier la valeur

`computed(...)`

Créer une valeur calculée

Les `Signals` sont aujourd'hui le mécanisme recommandé pour gérer l'état dans Angular moderne.

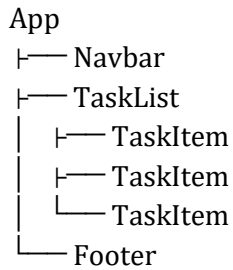
Angular pour développeur .NET - Partie 7

Communication entre composants : input() et output()

Pourquoi faire communiquer des composants ?

Une application Angular contient rarement un seul composant.

Exemple :



Les composants doivent pouvoir échanger des informations.

Parent et Enfant

Lorsqu'un composant contient un autre composant :



TaskList devient le parent.

TaskItem devient l'enfant.

Le problème

TaskList possède la liste complète des tâches.

TaskItem ne possède qu'une seule tâche.

Comment envoyer les informations du parent vers l'enfant ?

Comment prévenir le parent lorsqu'un bouton est cliqué dans l'enfant ?

La solution

Angular fournit deux outils :

input()
output()

input()

→ parent vers enfant

output()

→ enfant vers parent

Schéma général

Parent

|

| input()

v

Enfant

^

| output()

|

Parent

Comprendre input()

input() permet à un composant enfant de recevoir une donnée.

Exemple :

```
title = input.required<string>();
```

Le composant exige qu'un titre soit fourni.

Pourquoi required ?

required signifie :

'Cette donnée est obligatoire.'

Si le parent ne fournit pas la valeur,
Angular affiche une erreur.

Exemple complet input()

Enfant :

```
title = input.required<string>();
```

Template :

```
<p>{{ title() }}</p>
```

Parent :

```
<app-task-item  
  [title]="Apprendre Angular">  
</app-task-item>
```

Attention aux parenthèses

Comme les Signals :

title()

et non

title

C'est une erreur très fréquente.

Comprendre output()

output() permet à l'enfant d'envoyer un événement au parent.

L'enfant ne modifie pas directement les données du parent.

Il demande au parent d'agir.

Créer un output()

```
deleteRequested =  
  output<number>();
```

Ici l'événement renverra un nombre.

Envoyer une valeur

```
deleteRequested.emit(5);
```

Angular envoie la valeur 5 au parent.

Exemple complet output()

Enfant :

```
deleteRequested =  
  output<number>();
```

```
onDelete(){
  deleteRequested.emit(10);
}
```

HTML :

```
<button (click)="onDelete()">
  Supprimer
</button>
```

Recevoir l'événement dans le parent

Parent :

```
<app-task-item
  (deleteRequested)="onDelete($event)">
</app-task-item>
```

Le parent reçoit automatiquement la valeur.

Que contient \$event ?

\$event contient la valeur envoyée par emit().

```
emit(10)
```

→ \$event vaut 10

Exemple réel DevTask

TaskList contient :

```
tasks = [...]
```

TaskItem contient :

title

id

Quand l'utilisateur clique sur Supprimer :

TaskItem

→ emit(id)

TaskList

→ supprime la tâche

Pourquoi ne pas modifier directement le parent ?

Cela rendrait le code difficile à maintenir.

Angular préfère une communication claire :

Parent → Enfant

Enfant → Parent

Comparaison avec C#

input()

ressemble à un paramètre.

output()

ressemble à un événement.

Exemple C# :

```
button.Click += OnClick;
```

Le principe est très proche.

Exercice 1

Créer un composant UserCard.

Ajouter un input() name.

Exercice 2

Afficher le nom reçu dans le template.

Exercice 3

Créer un output() deleteRequested.

Déclencher emit() lors d'un clic.

Correction

```
name = input.required<string>();
```

```
deleteRequested =  
output<void>();
```

```
onDelete(){  
  deleteRequested.emit();  
}
```

Pièges fréquents

Oublier les parenthèses sur `input()`.

Oublier `emit()`.

Confondre `input` et `output`.

Faire modifier les données du parent par l'enfant.

Schéma récapitulatif

TaskList

|

| [title]

v

TaskItem

|

| emit(id)

v

TaskList

Résumé

`input()`

Recevoir une donnée

`output()`

Envoyer un événement

`emit()`

Déclencher l'événement

`$event`

Recevoir la valeur envoyée

Ces mécanismes sont indispensables dans toutes les applications Angular.

Angular pour développeur .NET - Partie 8

Les Services et l'Injection de Dépendances

Pourquoi les services existent ?

Dans une vraie application, plusieurs composants ont souvent besoin des mêmes données.

Exemple :

TaskList
TaskCounter
StatsPage

Tous ont besoin de connaître les tâches.

Si chaque composant stocke ses propres données, tout devient incohérent.

Le problème sans service

TaskList possède sa liste.
Stats possède sa liste.

Si une tâche est supprimée dans TaskList,
Stats ne le sait pas.

Les données deviennent désynchronisées.

La solution : les services

Un service est une classe spécialisée dans la logique métier.

Son rôle est de :

- stocker les données
- modifier les données
- appeler les API
- partager les données

Les composants affichent.
Les services travaillent.

Analogie du secrétariat

Imagine une entreprise.

Les employés (composants) ont besoin d'informations.

Ils ne stockent pas chacun leur propre copie.

Ils demandent au secrétariat (service).

Le service devient la source officielle des données.

Créer un service

Commande Angular :

```
ng g s task
```

ou

```
ng generate service task
```

Angular crée automatiquement le fichier :
task.service.ts

Structure d'un service

```
export class TaskService {  
  
}
```

Un service est simplement une classe TypeScript.

Injectable

```
@Injectable({  
  providedIn:'root'  
})
```

Cette instruction indique à Angular que le service doit être disponible dans toute l'application.

Que signifie providedIn: root ?

Angular crée une seule instance du service.

Tous les composants utilisent la même instance.

On parle de Singleton.

Pourquoi une seule instance ?

Si TaskList ajoute une tâche,
Stats doit voir cette nouvelle tâche.

Grâce à l'instance unique,
les données restent synchronisées.

Injection de dépendances

L'injection de dépendances permet à Angular de fournir automatiquement les objets nécessaires.

Tu n'as pas besoin d'utiliser new.

Sans injection

```
const service =  
  new TaskService();
```

Chaque composant créerait son propre service.

Avec injection

```
private taskService =  
  inject(TaskService);
```

Angular fournit automatiquement l'instance existante.

Pourquoi inject() ?

inject() est la méthode moderne recommandée.

Elle remplace souvent l'injection par constructeur.

Plus simple et plus lisible.

Comparaison avec ASP.NET Core

ASP.NET :

```
builder.Services.AddScoped<>()
```

Angular :

```
providedIn:'root'
```

Le principe est similaire.

Utiliser un service

```
taskService.addTask('Angular');
```

Le composant demande au service de faire le travail.

Service et Signals

Les Signals vivent souvent dans les services.

Exemple :

```
tasks = signal([]);
```

Tous les composants partagent alors le même Signal.

Exemple complet

```
@Injectable({
  providedIn:'root'
})
export class TaskService {

  tasks = signal([]);

  addTask(title:string){
  }

  deleteTask(id:number){
  }

}
```

Architecture recommandée

Component

↓

Service

↓

API

Le composant ne doit jamais appeler directement l'API.

Cycle complet

Utilisateur clique
↓
Composant
↓
Service
↓
Signal modifié
↓
Angular met à jour l'écran

Exercice 1

Créer un TaskService.

Exercice 2

Ajouter un Signal tasks.

Exercice 3

Ajouter une méthode addTask().

Exercice 4

Injecter TaskService dans TaskList.

Correction

```
private taskService =  
  inject(TaskService);
```

```
tasks =  
  this.taskService.tasks;
```

Erreurs fréquentes

Utiliser new TaskService().

Mettre toute la logique dans le composant.

Créer plusieurs sources de données.

Ne pas utiliser les services pour les appels API.

Quand créer un service ?

Lorsque plusieurs composants ont besoin :

- des mêmes données
- des mêmes calculs
- des mêmes appels API

C'est généralement le bon moment pour créer un service.

Résumé

Service :

Logique métier

`inject()`

Injection de dépendances

`providedIn:'root'`

Instance unique

Les services constituent la colonne vertébrale d'une application Angular bien organisée.

Comparaison avec C#

C# :

```
await client.GetAsync()
```

Angular :

```
http.get(...)  
.subscribe(...)
```

Le principe est similaire.

Cycle complet d'une requête

Composant



Service



HttpClient



API



Réponse JSON



Signal



Mise à jour écran

Pourquoi utiliser un service ?

Le composant ne devrait jamais appeler directement une API.

Bonne pratique :

Composant



Service



API

Exemple avec un Signal

```
tasks = signal([]);
```

```
loadTasks(){
```

```
this.http.get(...)
  .subscribe(data => {
    this.tasks.set(data);
  });
}
```

JSONPlaceholder

JSONPlaceholder est une API gratuite utilisée pour apprendre.

Exemple :

<https://jsonplaceholder.typicode.com/todos>

GET dans DevTask

TaskService

```
loadTasks(){

  this.http.get(...)
    .subscribe(data => {
      this.tasks.set(data);
    });
}
```

Quand charger les données ?

Le meilleur endroit est généralement `ngOnInit()`.

Le composant est complètement initialisé.

Exemple `ngOnInit`

```
ngOnInit(){

  this.taskService.loadTasks();

}
```

Autres méthodes HTTP

GET

Lire

POST

Créer

PUT

Modifier

DELETE

Supprimer

POST

Créer une ressource.

`http.post(url,data)`

PUT

Modifier une ressource existante.

`http.put(url,data)`

DELETE

Supprimer une ressource.

`http.delete(url)`

Erreurs fréquentes

Appeler l'API depuis le composant.

Oublier `subscribe()`.

Oublier `provideHttpClient()`.

Ne pas typer les données.

Questions d'examen fréquentes

À quoi sert `HttpClient` ?

Pourquoi utilise-t-on `subscribe()` ?

Quelle différence entre GET et POST ?

Pourquoi passer par un service ?

Exercice 1

Créer un service `UserService`.

Exercice 2

Injecter HttpClient.

Exercice 3

Créer une méthode loadUsers().

Correction

```
private http =  
  inject(HttpClient);  
  
loadUsers(){  
  
  this.http.get<User[]>(url)  
    .subscribe(data => {  
  
    });  
  
  }  
}
```

Résumé

HttpClient :

Communication avec une API

GET :

Lire

POST :

Créer

PUT :

Modifier

DELETE :

Supprimer

subscribe() :

Réagir lorsque les données arrivent

Les appels API constituent la passerelle entre Angular et les données réelles.

Angular pour développeur .NET - Partie 11

Projet complet DevTask : assembler toutes les briques

Pourquoi faire un projet complet ?

Jusqu'ici, chaque notion a été étudiée séparément.

Dans un projet réel, toutes ces notions travaillent ensemble.

L'objectif de DevTask est de comprendre comment Angular fonctionne dans son ensemble.

Architecture finale du projet

```
App
|
├─ Navbar
|
├─ TaskListPage
|   ├─ TaskItem
|   ├─ TaskItem
|   └─ TaskItem
└─ StatsPage
```

```
TaskService
|
├─ tasks
├─ addTask()
├─ deleteTask()
├─ toggleDone()
└─ remainingCount
```

Étape 1 : Créer le projet

```
ng new devtask
```

```
cd devtask
```

```
ng serve
```

Angular crée toute la structure de base.

Étape 2 : Créer les composants

```
ng g c pages/task-list
```

```
ng g c pages/stats
```

```
ng g c components/task-item
```

```
ng g c components/navbar
```

Étape 3 : Créer le service

```
ng g s services/task
```

Le service centralisera toutes les données.

Le modèle Task

```
type Task = {  
  id:number;  
  title:string;  
  done:boolean;  
};
```

Le Signal principal

```
tasks = signal<Task[]>([]);
```

Toutes les tâches seront stockées ici.

Le Computed

```
remainingCount = computed(() =>  
  this.tasks()  
    .filter(t => !t.done)  
    .length  
);
```

Il calcule automatiquement le nombre de tâches restantes.

Ajouter une tâche

```
addTask(title:string){  
  
  this.tasks.update(current => [  
    ...current,  
    {  
      id: Date.now(),  
      title,  
      done:false  
    }  
  ])  
};
```

```
}
```

Supprimer une tâche

```
deleteTask(id:number){  
  
  this.tasks.update(current =>  
    current.filter(t => t.id !== id)  
  );  
  
}
```

Changer l'état d'une tâche

```
toggleDone(id:number){  
  
  this.tasks.update(current =>  
    current.map(task =>  
      task.id === id  
      ? {...task, done: !task.done}  
      : task  
    )  
  );  
  
}
```

Le composant TaskItem

Responsabilité :

Afficher UNE seule tâche.

Il ne gère pas toute la liste.

Inputs du TaskItem

```
id = input.required<number>();
```

```
title = input.required<string>();
```

```
done = input.required<boolean>();
```

Outputs du TaskItem

```
deleteRequested =  
output<number>();
```

```
toggleRequested =  
output<number>();
```

HTML du TaskItem

```
<p>{{ title() }}</p>
```

```
<button  
(click)="toggleRequested.emit(id())">  
  Basculer  
</button>
```

```
<button  
(click)="deleteRequested.emit(id())">  
  Supprimer  
</button>
```

Le composant TaskList

TaskList récupère les données depuis TaskService.

Il affiche toutes les tâches.

Injection du service

```
private taskService =  
inject(TaskService);
```

```
tasks =  
this.taskService.tasks;
```

Boucle d'affichage

```
@for(task of tasks(); track task.id){
```

```
<app-task-item  
[id]="task.id"  
[title]="task.title"  
[done]="task.done" />
```

```
}
```

Gestion de la suppression

```
onDelete(id:number){
```



```
this.taskService.deleteTask(id);  
  
}
```

Gestion du changement d'état

```
onToggle(id:number){  
  
  this.taskService.toggleDone(id);  
  
}
```

Page statistiques

StatsPage utilise exactement le même service.

Elle affiche :

```
remainingCount()
```

Pourquoi ça fonctionne ?

Grâce à l'injection de dépendances.

Tous les composants partagent la même instance de TaskService.

Configuration des routes

```
export const routes = [  
  
  {  
    path:"",  
    component:TaskListPage  
  },  
  
  {  
    path:'stats',  
    component:StatsPage  
  }  
  
];
```

RouterOutlet

```
<router-outlet />
```

Angular affichera automatiquement la page correspondant à l'URL.

Navigation

```
<a routerLink="/">Tâches</a>
```

```
<a routerLink="/stats">  
  Statistiques  
</a>
```

Connexion à une API

```
private http =  
  inject(HttpClient);
```

```
loadTasks(){  
  
  this.http.get<Task[]>(url)  
    .subscribe(data => {  
  
      this.tasks.set(data);  
  
    });  
  
}
```

Chargement initial

```
ngOnInit(){  
  
  this.taskService.loadTasks();  
  
}
```

Cycle complet d'une suppression

Utilisateur clique



TaskItem



output.emit(id)



TaskList



TaskService.deleteTask()



Signal mis à jour



Computed recalculé



HTML mis à jour

Ce que ce projet utilise

- ✓ TypeScript
- ✓ Composants
- ✓ Binding
- ✓ @if
- ✓ @for
- ✓ Signals
- ✓ Computed
- ✓ Input
- ✓ Output
- ✓ Services
- ✓ Injection de dépendances
- ✓ Routing
- ✓ HttpClient

Questions fréquentes d'examen

Pourquoi utiliser un service ?

Pourquoi utiliser track ?

Pourquoi utiliser routerLink ?

Pourquoi utiliser computed() ?

Pourquoi utiliser input() ?

Résumé final

DevTask est un projet complet permettant de mettre en pratique toutes les notions Angular modernes.

Si tu comprends DevTask, tu comprends déjà l'architecture d'une grande partie des applications Angular professionnelles.

Angular pour développeur .NET - Partie 12

Préparation Examen Angular

Cette partie regroupe les questions les plus probables, les pièges fréquents et une fiche mémo à relire avant une évaluation.

1. Qu'est-ce qu'Angular ?

Un framework frontend permettant de créer des applications web organisées en composants.

2. Qu'est-ce qu'un composant ?

Une partie réutilisable de l'interface utilisateur.

3. Quel est le rôle de main.ts ?

Démarrer l'application Angular.

4. À quoi sert AppComponent ?

C'est le composant racine de l'application.

5. Différence entre framework et bibliothèque ?

Un framework impose une structure. Une bibliothèque est un simple outil.

6. À quoi sert TypeScript ?

Ajouter du typage pour détecter les erreurs avant l'exécution.

7. Différence entre type et interface ?

Les deux décrivent la structure d'un objet. type est souvent plus flexible.

8. À quoi sert {{ }} ?

Interpolation : afficher une valeur.

9. À quoi sert [] ?

Property Binding : envoyer une valeur au HTML.

10. À quoi sert () ?

Event Binding : écouter un événement.

11. À quoi sert @if ?

Afficher un bloc sous condition.

12. À quoi sert @for ?

Afficher une collection.

13. Pourquoi utiliser track ?

Pour aider Angular à identifier les éléments d'une liste.

14. À quoi sert @switch ?

Gérer plusieurs cas possibles.

15. Qu'est-ce qu'un Signal ?

Une valeur réactive qui prévient Angular lorsqu'elle change.

16. Comment lire un Signal ?

Avec des parenthèses : count().

17. Différence entre set() et update() ?

set remplace la valeur. update calcule une nouvelle valeur depuis l'ancienne.

18. À quoi sert computed() ?

Créer une valeur calculée automatiquement.

19. Peut-on modifier un computed ?

Non.

20. À quoi sert input() ?

Recevoir une donnée du parent.

21. À quoi sert output() ?

Envoyer un événement au parent.

22. À quoi sert emit() ?

Déclencher un output.

23. Que contient \$event ?

La valeur envoyée par emit().

24. Pourquoi utiliser un service ?

Partager une logique ou des données entre composants.

25. À quoi sert inject() ?

Récupérer un service via l'injection de dépendances.

26. Que signifie providedIn:'root' ?

Une seule instance du service dans toute l'application.

27. À quoi sert RouterLink ?

Naviguer sans recharger la page.

28. À quoi sert RouterOutlet ?

Afficher le composant correspondant à la route.

29. Pourquoi éviter href ?

Il recharge complètement la page.

30. À quoi sert HttpClient ?

Communiquer avec une API.

Les pièges préférés des formateurs

- Confondre input() et output().
- Oublier les parenthèses d'un Signal.
- Oublier track dans @for.
- Utiliser href au lieu de routerLink.
- Modifier un tableau de Signal avec push().
- Mettre les appels API dans les composants au lieu des services.
- Utiliser set() alors qu'update() serait plus adapté.
- Oublier provideHttpClient().
- Oublier provideRouter().
- Confondre Signal et Computed.

Fiche mémo express

{{ }} = afficher

[property] = envoyer une valeur

(event) = écouter un événement

@if = condition

@for = boucle

@switch = plusieurs cas

signal() = créer un Signal

signal() = lire un Signal

signal.set() = remplacer une valeur

signal.update() = modifier une valeur

computed() = valeur calculée

input() = parent → enfant

output() = enfant → parent

inject() = récupérer un service

routerLink = naviguer

router-outlet = afficher une page

HttpClient = appeler une API